

Grafo

Cos'è: È una coppia di nodi e relazioni $G = (V, E)$ $E \subset V^2$

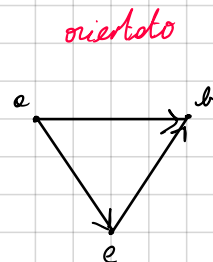
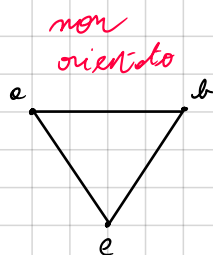
coppia $\rightarrow$ ^(V, U) ordinate = Grafo Orientato / Diretto / Digrafo
inferiamo questa

non ordinate = Grafo non orientato
 $\{u, v\} = \{v, u\}$

$$(u, v) = (v, u) \quad (u, v) \in E \Rightarrow (v, u) \in E$$

Rappresentazione grafica grafo: $V = \{a, b, c\}$

$$E = \{(a, b), (a, c), (b, c)\}$$



Cosa sono gli adiacenti: $A(u) = \{v \in V \mid (u, v) \in E\}$

orientato: $A(a) = \{b, c\}$ $A(b) = \{c\}$

non orientato: $A(a) = \{b, c\}$ $A(b) = \{a, c\}$

Cammino: cammino da $u, v = v_1, v_2, \dots, v_l$ t.e. $u = v_0, v_l$ $\forall i \in [0, l): (v_i, v_{i+1}) \in E$ $lunghezza = l$

$\cup \rightsquigarrow$

*2mo cammino /
V è raggiungibile da U*

quando esiste cammino di lunghezza ≥ 1 da u a u , questo è un ciclo

un grafo è aciclico se non esistono cicli

grafo connesso: $\forall u, v \in V \quad u \neq v \quad \text{ho che } u \rightsquigarrow v \text{ oppure } v \rightsquigarrow u$

6. fortemente connesso: $\forall u, v \in V \quad u \neq v \quad \text{ho che } u \rightsquigarrow v$

un grafo fortemente connesso è ciclico

8. denso/sperso: **denso** = ha quasi tutti gli archi

$$|E| = \Theta(|V|^2)$$

↑ cardinalità di V

sperso = ha pochi archi

$$|E| = \Theta(|V|)$$

Formalizzazione grafi: Abbiamo 2 possibili approcci:

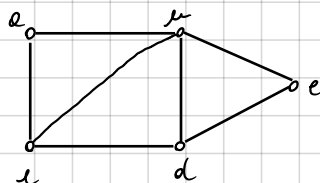
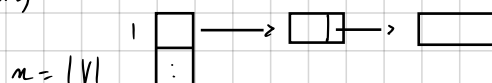
1) Liste di adiacenza: Rappresentiamo il grafo come un array con $|V|$ elementi

$$\text{Spazio occupato} = \Theta(n+m)$$

$n = \text{modi}$

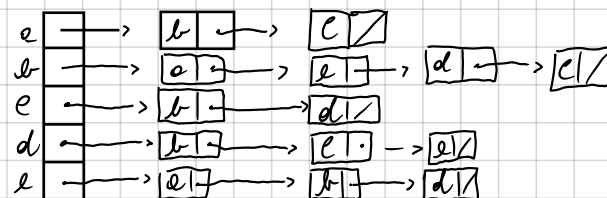
$m = \text{archi}$

proprio per i grafi densi



Lista adiacenza = $A(v_i)$

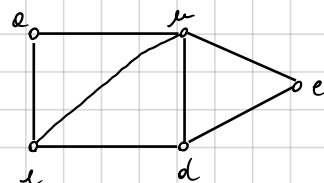
Analisi ordine degli adiacenti è ammissibile



2) Matrice di adiacenza: Un grafo rappresentato attraverso una matrice binaria $n \times n$ M t.e. $M[i,j] = 1 \Leftrightarrow (v_i, v_j) \in E$

$$\text{Spazio occupato} = \Theta(n^2)$$

proprio per i grafi sparsi



	a	b	c	d	e
a	0	1	0	0	1
b	1	0	1	1	1
c	0	1	0	1	0
d	0	1	1	0	1
e	1	1	0	1	0

Problema: Distanza da una sorgente

Input: Grafo $G = (V, E)$, $s = v$

Output: $\forall v \in V$ distanza da s a v

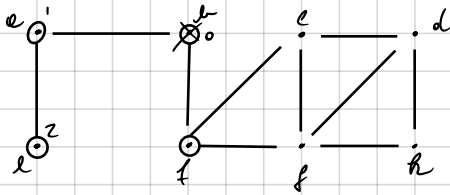
distanza = lunghezza cammino minimo più corto

$\forall v \in V: s \rightarrow v \quad d(v) = +\infty$

Strategie: 1) visita FIFO $\Rightarrow$ code

2) scoperti / visitati $\rightarrow$ colori $\rightarrow$ $\begin{cases} W = \text{non scoperto} \\ G = \text{scoperto ma non visitato} \\ B = \text{visitato} \end{cases}$

3) tenere traccia ~~per le~~ ~~colori~~



$Q =$

b	
---	--

a.

o scoperto

X visitato

Algoritmo BFS / Ricerca in ampiezza

Cos'è? È una tecnica usata per esplorare un grafo a partire da un nodo sorgente, visitando prima i nodi vicini (in termini di archi), poi quelli più lontani. È particolarmente utile per trovare le distanze minime (in numero di archi) da un nodo sorgente a tutti gli altri nodi di

Strategie: 1) Parti da un nodo sorgente

2) Visita tutti i vicini immediati

3) Poi visita i vicini dei vicini

4) Usa una code (FIFO) per tenere traccia dei nodi da esplorare

5) Ogni nodo viene esplorato una sola volta, e in ordine di distanze crescente dalla sorgente

Colori di un nodo:

- Bianco = nodo non ancora visitato
- Grigio = Nodo in code per essere esplorato
- Nero = Nodo completamente esplorato

Analisi dei tempi

V = numero di nodi
 M = numero di archi

Passaggi: 1) Inizializzazione di ogni nodo = $\Theta(V)$

2) Visita di ogni nodo e arco = $\Theta(V+M)$

$$T = \Theta(V+M)$$

procedura BFS (Grafo G, Nodo s)

// inizializzazione di ogni nodo del grafo

for each $v \in G$

$v.colore = Bianco$

$v.distancia = +\infty$

$v.predecessore = null$

$\Theta(n)$

// inizializzo la sorgente

$s.colore = grigio$

$s.distancia = 0$ // la distanza tra un nodo e se stesso è 0

$s.predecessore = null$

$\Theta(1)$

// Code per l'esplorazione in ampiezza

Code Q = nuova code

Enqueue(Q, s)

// Ciclo principale: esplora i nodi finché la code è piena

while ! Q - E

$v = Dequeue(Q)$

// Esamine tutti i vicini di v

for each $u \in A$

if $u.colore == Bianco$

$u.colore = grigio$

$u.distancia = v.distancia + 1$

$u.predecessore = v$

Enqueue(Q, u)

$v.colore = Nero$

$\Theta(n+m)$

procedura stampa_percorso (G, v)

// v: nodo di destinazione da cui risalire al percorso

// Caso 1: se v non è stato raggiunto durante la BFS

if $v.distancia == +\infty$

print("nessun percorso")

// Caso 2: se v ha un predecessore (quindi non è la sorgente)

else if $v.predecessore \neq null$

// Prima stampa il percorso fino al predecessore

stampa_percorso(G, v.predecessore)

print(v)

// Caso 3: v è la sorgente (non ha predecessore)

else

print(v)

Marca in profondità

Strategie:

- Colori:
- Bianco = non ancora scoperto
 - Grigio = scoperto e sto visitando
 - Nero = visitato completamente (ho visitato tutti i nodi raggiungibili da questo nodo)

Predecessori:

- Tempi di visita:
- $u.d$ = tempo di scoperta
 - $u.f$ = tempo di fine visita
(tempo imputato da Grigio $\rightarrow$ Nero)
- I tempi variano da 1 a $2 * |V|$

procedure DFS(G)

// inizializzo i nodi

for each $v \in V(G)$ do

$v.color = W$

$v.parent = null$

$\Theta(n)$

// Inizializzo il tempo

time = 0

$\Theta(1)$

// Visito i vari nodi

for each $v \in V(G)$ do

if $v.color == W$ then

DFS_visit(G, v)

$\Theta(n)$

Tempo = $\Theta(n + m)$

Classificazione Orchi: $(u, v) \in E$

1) Tree Edge $v.parent = u$ v era bianco quando sto visitando u

2) Back Edge mentre visito u , v era figlio

3) Cross Edge mentre visito u , v è nero

4) Forward Edge mentre visito u , v è nero
 $v.d < v.f < u.f$
 $u.d < v.d < v.f < u.f$

procedure DFS_visit(G, v)

// "inizializzo" il nodo

$v.color = G$

time = time + 1

$v.d = time$

$\Theta(n)$ in totale

// esploro il nodo

for each $u \in Adj(v)$ do

if $u.color == W$ then

$u.parent = v$

DFS_visit(G, u)

$\Theta(m)$ m è il numero di archi

// aggiorno lo stato del nodo

$v.color = B$

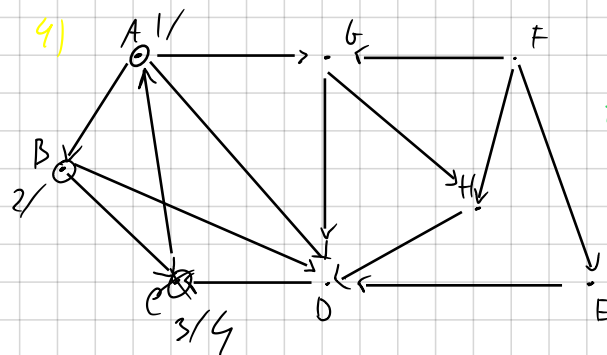
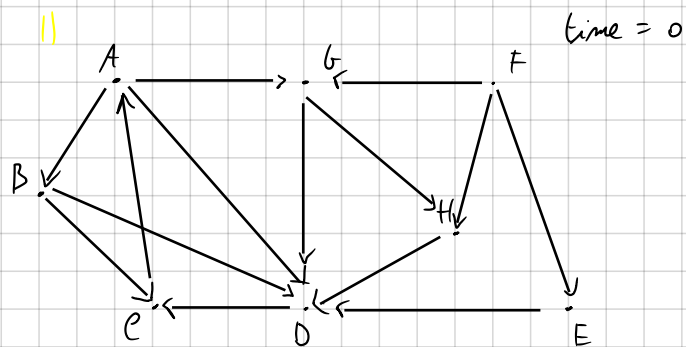
time = time + 1

$v.f = time$

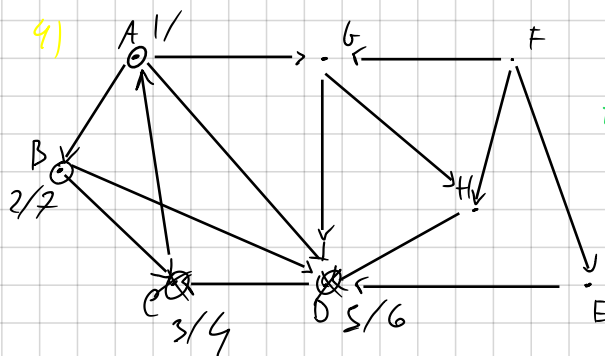
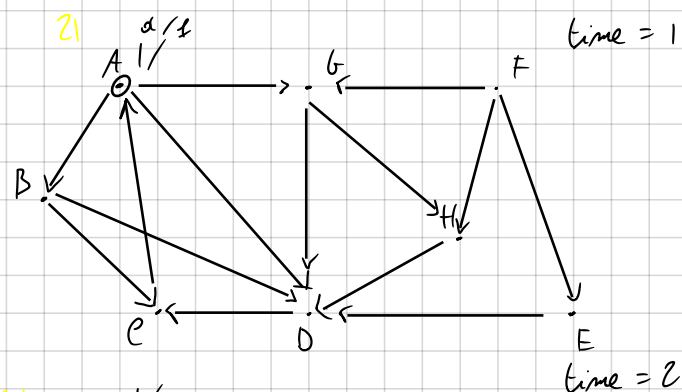
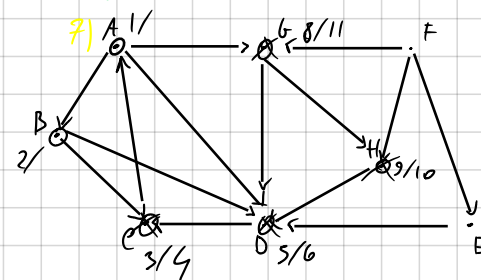
$\Theta(n)$ in totale

esempio Grafico

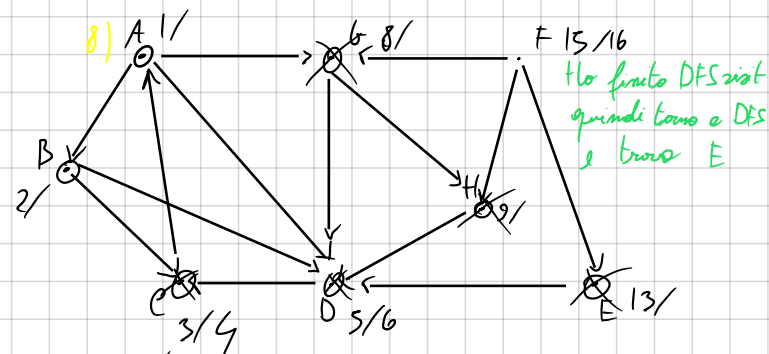
0 = Grigio
X = Nero



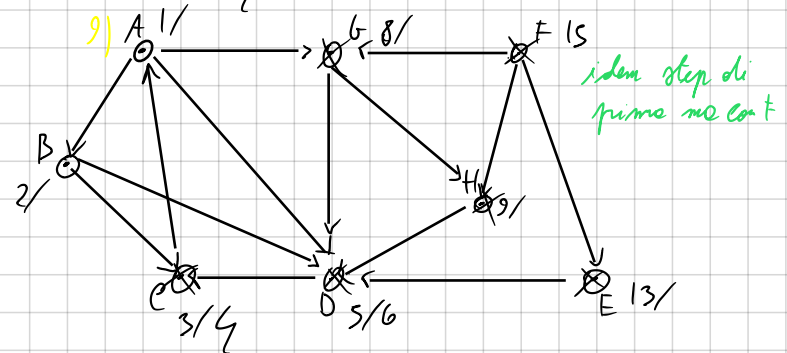
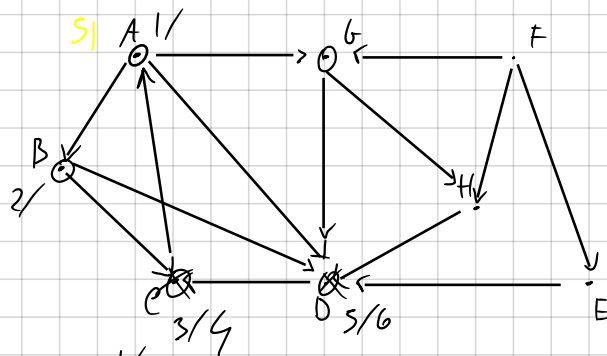
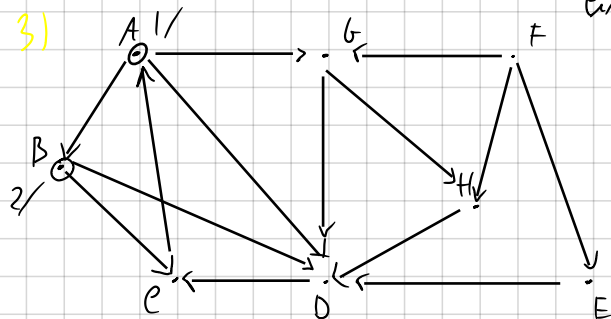
ho finito e quindi torno a D



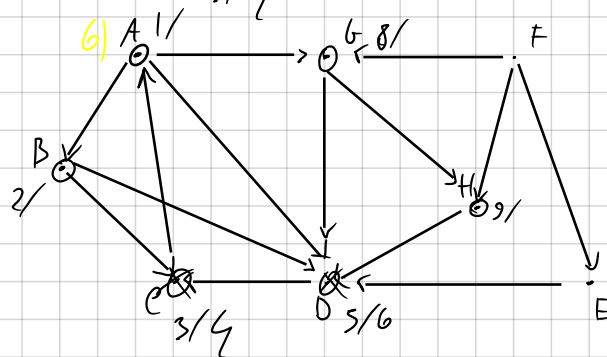
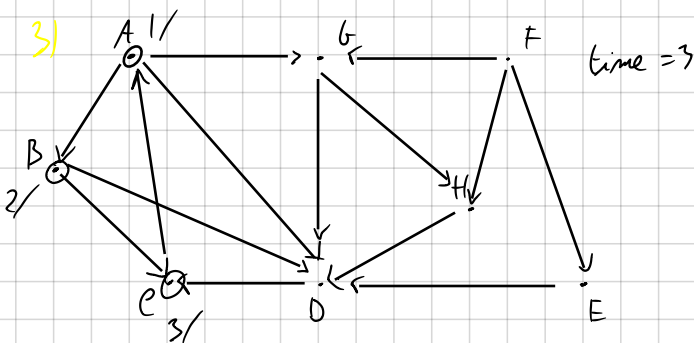
ho finito e quindi torno a D



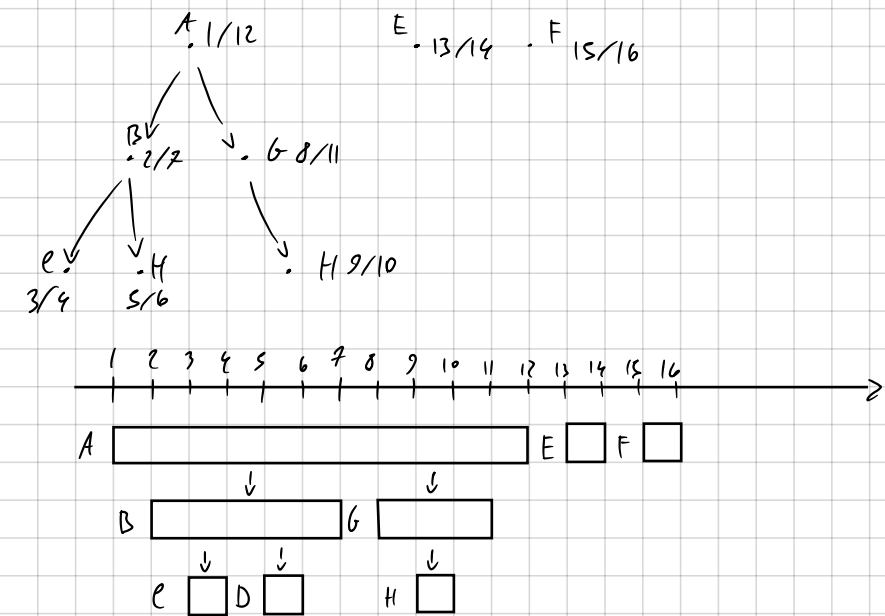
Ho finito DFS visit quindi torno a DFS e torno E



idem step di prima ma con F



Allen qudocaporo



I tempi di inizio / fine visita rispettano la struttura parentesi

Struttura Parentesi: $\forall u, v \in V$ uno e soltanto un caso può avvenire

- 1) v è discendente di u e $u.d < v.d < v.f < u.f$
- 2) u è discendente di v e $v.d < u.d < u.f < v.f$
- 3) né u è discendente di v né v è discendente di u e $[u.d, u.f]$ e $[v.d, v.f]$ sono disgiunti

Corollario: u è discendente di v se e solo se $v.d < u.d < u.f < v.f$

Classificazione Orchi: $(u, v) \in E$

- 1) Tree Edge $v.parent = u$ v era bianco quando stava visitando u
- 2) Back Edge mentre visita u , v era grigio
- 3) Cross Edge mentre visita u , v è nero
- 4) Forward Edge mentre visita u , v è nero

Lemma: G è aciclico se DFS non incontra Back Edge.

Ordinamento topologico: $v_1, v_2, \dots, v_n$ perm di V t.e. $\forall (v_i, v_j) \in E \Rightarrow i < j$